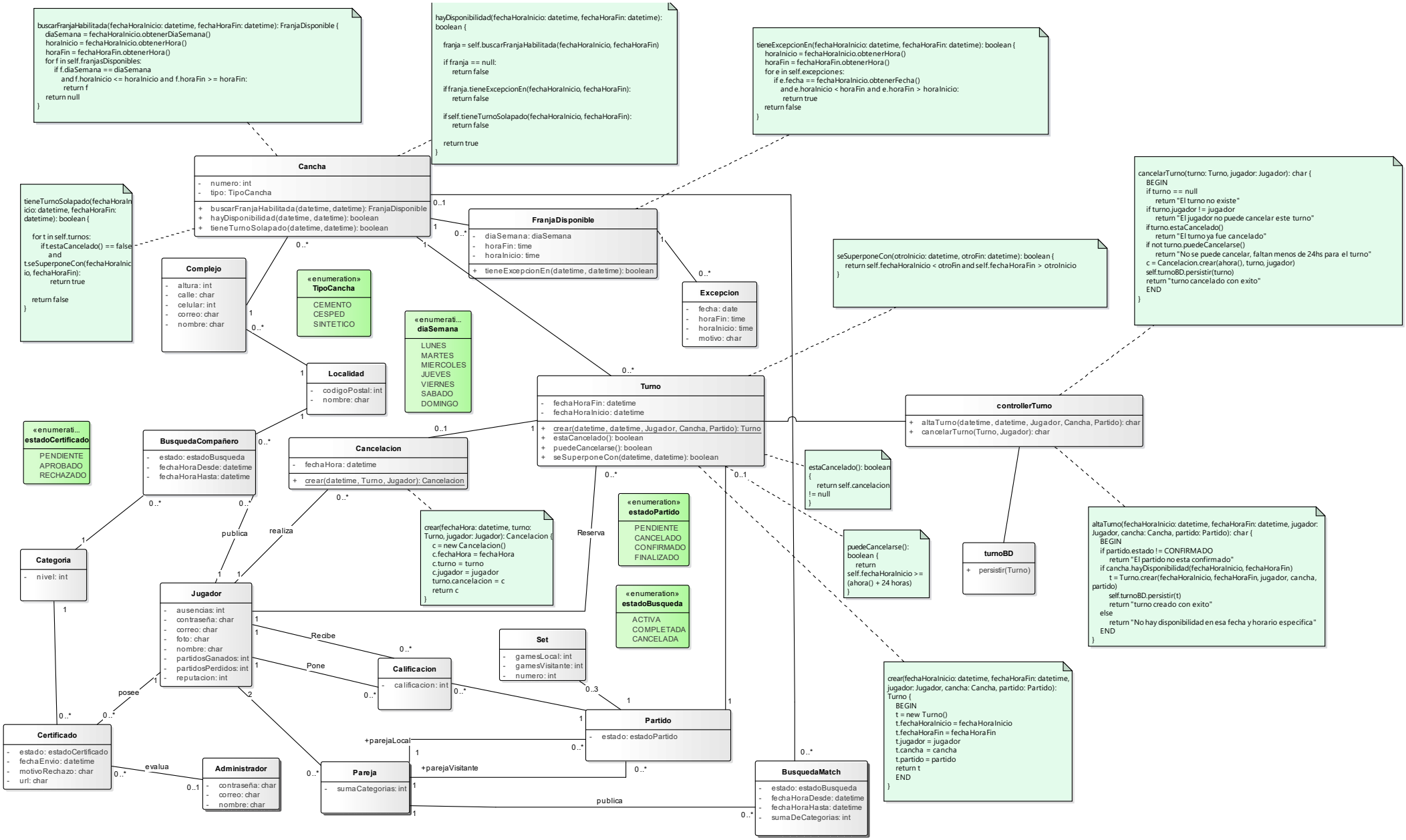

buscarFranjaHabilitada(fechaHoraInicio: datetime, fechaHoraFin: datetime): FranjaDisponible {
diaSemana = fechaHoraInicio.obtenerDiaSemana()
horaInicio = fechaHoraInicio.obtenerHora()
horaFin = fechaHoraFin.obtenerHora()
for f in self.franjasDisponibles:
if f.diaSemana == diaSemana
and f.horaInicio <= horaInicio and f.horaFin >= horaFin:
return f
return null
}
hayDisponibilidad(fechaHoraInicio: datetime, fechaHoraFin: datetime): boolean {
franja = self.buscarFranjaHabilitada(fechaHoraInicio, fechaHoraFin)
if franja == null:
return false
if franja.tieneExcepcionEn(fechaHoraInicio, fechaHoraFin):
return false
if self.tieneTurnoSolapado(fechaHoraInicio, fechaHoraFin):
return false
return true
}
tieneExcepcionEn(fechaHoraInicio: datetime, fechaHoraFin: datetime): boolean {
horaInicio = fechaHoraInicio.obtenerHora()
horaFin = fechaHoraFin.obtenerHora()
for e in self.excepciones:
if e.fecha == fechaHoraInicio.obtenerFecha()
and e.horaInicio < horaFin and e.horaFin > horaInicio:
return true
return false
}
tieneTurnoSolapado(fechaHoraInicio: datetime, fechaHoraFin: datetime): boolean {
for t in self.turnos:
if t.estaCancelado() == false
and t.seSuperponeCon(fechaHoraInicio, fechaHoraFin):
return true
return false
}
seSuperponeCon(otroInicio: datetime, otroFin: datetime): boolean {
return self.fechaHoraInicio < otroFin and self.fechaHoraFin > otroInicio
}
cancelarTurno(turno: Turno, jugador: Jugador): char {
BEGIN
if turno == null
return "El turno no existe"
if turno.jugador != jugador
return "El jugador no puede cancelar este turno"
if turno.estaCancelado()
return "El turno ya fue cancelado"
if not turno.puedeCancelarse()
return "No se puede cancelar, faltan menos de 24hs para el turno"
c = Cancelacion.crear(ahora(), turno, jugador)
self.turnoBD.persistir(turno)
return "turno cancelado con exito"
END
}
Cancha
- numero: int
- tipo: TipoCancha
+ buscarFranjaHabilitada(datetime, datetime): FranjaDisponible
+ hayDisponibilidad(datetime, datetime): boolean
+ tieneTurnoSolapado(datetime, datetime): boolean
FranjaDisponible
- diaSemana: diaSemana
- horaFin: time
- horaInicio: time
+ tieneExcepcionEn(datetime, datetime): boolean
Excepcion
- fecha: date
- horaFin: time
- horaInicio: time
- motivo: char
Complejo
- altura: int
- calle: char
- celular: int
- correo: char
- nombre: char
«enumeration»
TipoCancha
CEMENTO
CESPED
SINTETICO
«enumerati...
diaSemana
LUNES
MARTES
MIERCOLES
JUEVES
VIERNES
SABADO
DOMINGO
Localidad
- codigoPostal: int
- nombre: char
Turno
- fechaHoraFin: datetime
- fechaHoraInicio: datetime
+ crear(datetime, datetime, Jugador, Cancha, Partido): Turno
+ estaCancelado(): boolean
+ puedeCancelarse(): boolean
+ seSuperponeCon(datetime, datetime): boolean
controllerTurno
+ altaTurno(datetime, datetime, Jugador, Cancha, Partido): char
+ cancelarTurno(Turno, Jugador): char
«enumerati...
estadoCertificado
PENDIENTE
APROBADO
RECHAZADO
BusquedaCompañero
- estado: estadoBusqueda
- fechaHoraDesde: datetime
- fechaHoraHasta: datetime
Cancelacion
- fechaHora: datetime
+ crear(datetime, Turno, Jugador): Cancelacion
estaCancelado(): boolean {
return self.cancelacion != null
}
puedeCancelarse(): boolean {
return self.fechaHoraInicio >= (ahora() + 24 horas)
}
turnoBD
+ persistir(Turno)
altaTurno(fechaHoraInicio: datetime, fechaHoraFin: datetime, jugador: Jugador, cancha: Cancha, partido: Partido): char {
BEGIN
if partido.estado != CONFIRMADO
return "El partido no esta confirmado"
if cancha.hayDisponibilidad(fechaHoraInicio, fechaHoraFin)
t = Turno.crear(fechaHoraInicio, fechaHoraFin, jugador, cancha, partido)
self.turnoBD.persistir(t)
return "turno creado con exito"
else
return "No hay disponibilidad en esa fecha y horario especifica"
END
}
crear(fechaHora: datetime, turno: Turno, jugador: Jugador): Cancelacion {
c = new Cancelacion()
c.fechaHora = fechaHora
c.turno = turno
c.jugador = jugador
turno.cancelacion = c
return c
}
«enumeration»
estadoPartido
PENDIENTE
CANCELADO
CONFIRMADO
FINALIZADO
«enumeration»
estadoBusqueda
ACTIVA
COMPLETADA
CANCELADA
Categoria
- nivel: int
Jugador
- ausencias: int
- contraseña: char
- correo: char
- foto: char
- nombre: char
- partidosGanados: int
- partidosPerdidos: int
- reputacion: int
Set
- gamesLocal: int
- gamesVisitante: int
- numero: int
Calificacion
- calificacion: int
Partido
- estado: estadoPartido
crear(fechaHoraInicio: datetime, fechaHoraFin: datetime, jugador: Jugador, cancha: Cancha, partido: Partido): Turno {
BEGIN
t = new Turno()
t.fechaHoraInicio = fechaHoraInicio
t.fechaHoraFin = fechaHoraFin
t.jugador = jugador
t.cancha = cancha
t.partido = partido
return t
END
}
Certificado
- estado: estadoCertificado
- fechaEnvio: datetime
- motivoRechazo: char
- url: char
Administrador
- contraseña: char
- correo: char
- nombre: char
Pareja
- sumaCategorias: int
BusquedaMatch
- estado: estadoBusqueda
- fechaHoraDesde: datetime
- fechaHoraHasta: datetime
- sumaDeCategorias: int
publica
realiza
Reserva
Recibe
Pone
posee
evalua
+parejaLocal
+parejaVisitante
publica